

Practice 4:

Deployment and Interactive Demonstrations

Course: Software Development Oriented to Machine Learning

Overview

In this final practice, you'll deploy your ML project as an interactive demonstration, making it accessible to users who want to explore your work without diving into code. You'll create a user-friendly interface that showcases your data, training capabilities, and model evaluation. This practice transforms your project from a development artifact into a deployable application.

Objectives

Learn to create interactive ML demonstrations using tools like Gradio. Deploy applications to public platforms for easy access. Package ML projects for distribution and reusability. Transform technical projects into user-friendly applications that showcase your work.

Requirements

The requirements for this final practice are:

Interactive Demo: Create an interactive demonstration using [Gradio](#) (or equivalent tools like Streamlit). Your demo must include three main sections, organized in a clear interface (tabs, pages, or similar):

- **Data Exploration:** Allow users to visualize and explore your dataset. Include interactive plots, sample displays, and basic statistics. Users should be able to understand your data without reading code. Reuse (and extend if you can) your previous visualizations.
- **Training Interface:** Provide controls for training the model with different hyperparameters. Users should be able to adjust key parameters (learning rate, epochs, batch size, etc.) and initiate training. Display training progress and a summary of final metrics.

Obviously, if your training is too slow, you will only be able to offer a “preview” of it, with a reduced portion of the dataset, reduced number of iterations...

- **Model Evaluation:** Enable users to evaluate the trained model. Show performance visualizations, prediction examples, and relevant metrics. For classification tasks, include confusion matrices and sample predictions; for regression, show prediction vs. actual plots. Reuse (and

extend if you can) your previous visualizations. If the model is not extremely heavy, you can include or share a single checkpoint (your best version) so that it can be evaluated here, without doing any training. Alternatively, it's common to include only the *outputs* of the model for your test dataset, so that it can be evaluated even in absence of the checkpoints.

Deployment: Deploy your interactive demo using one of the following options:

- **PyPI Package:** Publish your project as a package on PyPI. Ensure the package is modular so users can import and reuse components. Include a command-line entry point that launches the demo locally (e.g., `uvx your_package_name` will launch it).
- **Hugging Face Spaces:** Deploy to [Hugging Face Spaces](#) as either a public or private space. This provides free hosting and easy sharing.
- **Docker Container:** Create a Docker container and publish it to Docker Hub. Include comprehensive instructions in your README for running the container locally.
- **Alternative Hosting** (Lower evaluation): Deploy using Google Colab or university-provided container instances. While acceptable, these options typically result in a lower final mark.

Data Availability: Consider the data availability challenge for deployment:

- Include mock/sample data crafted by yourself for immediate demonstration, OR
- Include the full dataset if the LICENSE allows for it and it's small enough, OR
- Implement an automatic download function that fetches the real data if not already present

In summary, ensure users can run your demo without manual data preparation steps, and without violating any data licenses constraints on redistribution.

Technical Specifications

Your interactive demo should be intuitive and self-explanatory. Include helpful descriptions, tooltips, and guidance for users unfamiliar with your project. The interface should handle errors gracefully and provide informative feedback during long-running operations like training.

Ensure all components of your demo import from the same underlying codebase. The training interface should use the exact same model and data processing code as your original training scripts. No code duplication between your demo and core project: use imports adequately.

For PyPI packages, structure your code to be modular and reusable. Users should be able to import specific components (data loaders, models, visualization functions) without launching the full demo. Include clear examples in your documentation.

Include deployment instructions in your README.md with step-by-step guidance for reproducing your deployment. Document any environment variables, API keys (use environment variables!), or configuration files needed.

Deliverables

Submit only:

Repository URL: Link to your updated repository containing the deployment code and documentation.

All code must be accessible through your public repository, with the README file including all necessary instructions, as well as links to the documentation. The demo should be functional at the time of evaluation.

Final Project Evaluation

This practice marks the completion of your bimester-long project. Your final grade will consider all five practices holistically, with emphasis on:

Code Organization and Modularity:

- Logical project structure following professional standards
- No code duplication between demo, training scripts, and utilities
- Proper separation of concerns (data loading, model definition, training, evaluation)
- All components importing from shared modules (especially data and model classes)

Documentation Quality:

- Complete README.md with step-by-step setup and execution instructions
- Working installation commands that actually reproduce the environment
- Clear data source information and acquisition instructions
- LICENSE file included in the repository
- Comprehensive inline documentation using consistent docstring style
- Professional HTML documentation generated from docstrings. Remember to update this documentation when you change things in the code
- No blank pages or missing module/submodule documentation in HTML output

Version Control Practices:

- Accessible public repository with a “realistic” commit history
- Evidence of multi-author collaboration with at least one feature branch properly merged into main
- Clear, descriptive commit messages
- Proper .gitignore configuration. No .venv or similar uploaded to the repo

Deployment and Usability:

- Functional interactive demo that works as specified
- Clear deployment strategy with accessible hosting
- User can launch the demo without extensive troubleshooting
- Data availability handled appropriately (mock data or auto-download)

Important Notes

Remember that a real user should be able to get your project running as quickly as possible. Every manual step you require is a barrier to adoption. Automate data downloads, provide clear dependency installation, and **test your deployment instructions on a fresh system.**

Your interactive demo is often the first (and sometimes only) thing potential employers, collaborators, or users will see. Make it count. A polished, working demo demonstrates professional software engineering skills beyond just ML knowledge.

Focus on creating something you'd be proud to share publicly. This project represents your work for an entire bimester and should showcase both your ML understanding and software development capabilities.

Technical Guidelines

Gradio Tips: Use `gr.Progress()` to show training progress or data download.

Data Handling: If implementing auto-download, use libraries like `huggingface_hub`, `kaggle`, or direct HTTP downloads. Cache downloaded data to avoid repeated downloads. Include file size warnings for large datasets. If an API key is needed, just prompt the user for one (never include API keys in your code).

Docker Considerations: Document required ports and volume mounts. Test your container on a clean system before submission.

PyPI Packaging: Use `pyproject.toml` for package configuration. Include your interactive demo as a console script entry point. Version your package semantically. Test installation in a fresh virtual environment.